

DoarFazBem - Frontend

Sistema de gestão de doações de sangue. Este frontend React + TypeScript é a interface do usuário para o ecossistema de doação, suporte a hemocentros, doadores e receptores.

Sumário

- [Tecnologias](#)
- [Instalação](#)
- [Execução](#)
- [Estrutura do Projeto](#)
- [Funcionalidades](#)
- [Perfis de Usuário](#)
- [API](#)
- [Internacionalização](#)
- [Organização de Componentes](#)
- [Boas práticas e evolução](#)

Tecnologias

- **Vite 8** - Build tool rápida e moderna.
- **React 19** - Biblioteca de UI.
- **TypeScript** - Tipagem segura.
- **Tailwind CSS 4** - Estilização utilitária.
- **React Router DOM 7** - Roteamento de páginas.
- **React Query (TanStack)** - Fetch, cache e sincronização de dados.
- **Axios** - Cliente HTTP.
- **React Hook Form** - Gestão de formulários.
- **Zod** - Validação de schemas.
- **i18next / react-i18next** - Internacionalização.
- **Lucide-react** - Ícones.
- **Sonner** - Toasts de notificação.

Instalação

Instale dependências:

```
npm install
```

Execução

Desenvolvimento:

```
npm run dev
```

Build de produção:

```
npm run build
```

Pré-visualização:

```
npm run preview
```

Variáveis de Ambiente

Crie `.env` na raiz com a URL do backend:

```
VITE_API_URL=http://localhost:8080
```

Estrutura do Projeto

```
src/
├── assets/                # Imagens, ícones e arquivos estáticos
├── components/           # Componentes reutilizáveis e específicos
│   ├── admin/           # Modais e formulários administrativos
│   │   ├── HistoricoModal.tsx
│   │   ├── TriagemModal.tsx
│   │   ├── TriagemTransfusaoModal.tsx
│   │   └── forms/       # Formulários específicos do admin
│   │       ├── AgendamentoForm.tsx
│   │       ├── CampanhaForm.tsx
│   │       ├── EstoqueSangueForm.tsx
│   │       ├── HemocentroForm.tsx
│   │       ├── PedidoTransfusaoForm.tsx
│   │       ├── PedidoTransfusaoFormSimple.tsx
│   │       ├── PerfilDoadorForm.tsx
│   │       └── UsuarioForm.tsx
│   ├── common/          # Componentes de UI genéricos
│   │   ├── Button.tsx
│   │   ├── Card.tsx
│   │   ├── DateInput.tsx
│   │   ├── Input.tsx
│   │   └── Select.tsx
│   └── layouts/          # Wrappers de rota e segurança
│       └── ProtectedRoute.tsx
├── contexts/             # Contextos React (auth, user)
│   └── AuthContext.tsx
├── hooks/                 # Hooks customizados
│   └── useApi.ts
├── i18n/                 # Configuração de tradução
│   └── config.ts
├── layouts/              # Layouts por tipo de usuário
│   ├── MainLayout.tsx
│   ├── admin/AdminLayout.tsx
│   ├── doador/DoadorLayout.tsx
│   └── receptor/ReceptorLayout.tsx
├── locales/              # Traduções em JSON
└── en.json
```

```
|   └─ pt.json
└─ pages/                                # Páginas públicas e privadas
    │   └─ Contacto.tsx
    │   └─ EstoquePublico.tsx
    │   └─ Landing.tsx
    │   └─ Login.tsx
    │   └─ Register.tsx
    │   └─ Sobre.tsx
    │   └─ admin/                        # Telas do painel administrativo
    │   └─ doador/                      # Telas do perfil de doador
    │   └─ receptor/                   # Telas do perfil de receptor
└─ services/                           # Configuração e instância Axios
    └─ api.ts
└─ types/                              # Tipos TypeScript globais
    └─ index.ts
└─ App.tsx                             # Rotas e estrutura da aplicação
└─ main.tsx                            # Ponto de entrada React
```

Funcionalidades

Autenticação e autorização

- Login e registro de usuários.
- Proteção de rotas privadas.
- Controle de acesso por perfil: administrador, doador e receptor.
- Persistência de sessão via token.

Internacionalização

- Suporte a português e inglês.
- Troca de idioma em tempo real.
- Traduções centralizadas em `src/locales`.

Layouts e navegação

- Layout público para landing page e informações institucionais.
- Layouts privados separados para admin, doador e receptor.
- Menu de navegação específico por perfil.
- Rodapé e barra superior compartilhados.

Formulários e validação

- Formulários construídos com `React Hook Form`.
- Validação de campos com `Zod`.
- Reutilização de inputs, selects e componentes de formulário.

Consumo de API

- Chamadas HTTP centralizadas em `src/services/api.ts`.
- Axios configurado para headers e interceptors.
- React Query para cache, refetch e estado assíncrono.

Perfis de Usuário

Administrador

- Dashboard com indicadores gerais.
- Gerenciamento de usuários e doadores.
- Gerenciamento de campanhas.
- Controle de estoque de sangue.
- Gestão de hemocentros.
- Acompanhamento de pedidos de transfusão.
- Visualização de notificações.

Doador

- Visualização do próprio dashboard.
- Agenda de doações.
- Acesso a campanhas.
- Histórico de doações.
- Consulta de hemocentros.
- Atualização de perfil.
- Notificações pessoais.

Receptor

- Dashboard de solicitações.
- Histórico de requisições e transfusões.
- Detalhes de pedidos de sangue.
- Consulta de hemocentros.
- Atualização de perfil.
- Notificações.

API (backend)

A aplicação se integra a um backend REST por meio da variável `VITE_API_URL`.

Endpoints principais

Autenticação

- `POST /autenticacao/login`
- `POST /autenticacao/regar`

Usuários / Perfis

- `GET /usuarios`
- `POST /usuarios`
- `PUT /usuarios/:id`

Doadores

- `GET /doadores`
- `GET /doadores/:id`
- `POST /doadores`
- `PUT /doadores/:id`

Campanhas

- `GET /campanhas`
- `GET /campanhas/:id`
- `POST /campanhas`

- `PUT /campanhas/:id`

Agendamentos

- `GET /agendamentos`
- `POST /agendamentos`

Doações

- `GET /doacoes`
- `POST /doacoes`

Estoque

- `GET /estoque`

Hemocentros

- `GET /hemocentros`

Notificações

- `GET /notificacoes`

Organização de Componentes

`src/components/common`

Componentes reutilizáveis de interface:

- `Button.tsx`
- `Card.tsx`
- `DateInput.tsx`
- `Input.tsx`
- `Select.tsx`

`src/components/admin`

Componentes específicos do painel administrativo:

- Modais e formulários para triagem.
- Formulários de cadastro e edição de campanhas, hemocentros e pedidos.

`src/components/layouts`

- `ProtectedRoute.tsx` : direciona usuários não autorizados para login.

`src/layouts`

Layouts por tipo de perfil:

- `MainLayout.tsx`
- `AdminLayout.tsx`
- `DoadorLayout.tsx`
- `ReceptorLayout.tsx`

Boas práticas e evolução

- Recomendado adicionar testes com `Vitest` ou `Jest`.

- Implementar refresh token para autenticação mais segura.
- Adicionar gráficos de indicadores com biblioteca tipo `Recharts` .
- Melhorar acessibilidade WCAG e navegação por teclado.
- Expandir internacionalização para mais idiomas.

Observações

- `React Query` melhora o desempenho com cache de requisições.
- `Tailwind` permite acelerar o desenvolvimento de UI responsiva.
- A estrutura do projeto separa domínios de forma clara: componentes, layouts, páginas, serviços e tipos.
- O frontend está preparado para evoluir com novos perfis e recursos sem grandes alterações na arquitetura.

`error="Erro"`

`/>`

```
### Card
```tsx
<Card title="Título" description="Descrição" footer={<div>Footer</div>}>
 Conteúdo
</Card>
```

## Forms com React Hook Form e Zod

Todos os formulários utilizam React Hook Form para gerenciamento de estado e Zod para validação em tempo real:

```
const { register, handleSubmit, formState: { errors } } = useForm({
 resolver: zodResolver(schema)
});
```

## Autenticação e Proteção de Rotas

Rotas protegidas utilizam o componente `ProtectedRoute` que verifica:

- Se o usuário está autenticado
- Se possui o role necessário (para admin)
- Redireciona para login se não autenticado

## Como Contribuir

1. Faça um fork do projeto
2. Crie uma branch para sua feature ( `git checkout -b feature/AmazingFeature` )
3. Commit suas mudanças ( `git commit -m 'Add some AmazingFeature'` )
4. Push para a branch ( `git push origin feature/AmazingFeature` )
5. Abra um Pull Request

**Licença**

MIT